Step 1: Install Required Libraries

What are we performing?

- Installing the required LangChain libraries.

Why are we performing it?

- These libraries are required to build the LangChain RAG pipeline.

> **Difference from Previous Notebook**
>
> - Added `langchain_huggingface` and `langchain_community`.
- Uses LangChain components instead of manual FAISS implementation.

In [1]:
```
# pip install langchain_huggingface
# !pip install langchain_community
```

Step 2: Import Required Libraries

What are we performing?

- Importing libraries for preprocessing, chunking, embeddings, vector database and LLM.

Why are we performing it?

- These libraries will be used throughout the RAG pipeline.

> **Difference from Previous Notebook**
>
> - `HuggingFaceEmbeddings` replaces `SentenceTransformer` for embedding generation.
> - `FAISS` is imported from LangChain.
> - `ChatGoogleGenerativeAI` is imported for future LLM integration.

In [2]:
```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_google_genai import ChatGoogleGenerativeAI
import spacy
```

```
c:\Users\LENOVO\AppData\Local\Programs\Python\Python314\Lib\site-packages\langchain_core\utils\pydantic.py:41:
UserWarning: Core Pydantic V1 functionality isn't compatible with Python 3.14 or greater.
  from pydantic.v1 import BaseModel as BaseModelV1
c:\Users\LENOVO\AppData\Local\Programs\Python\Python314\Lib\site-packages\tqdm\auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
C:\Users\LENOVO\AppData\Local\Temp\ipykernel_19472\933101058.py:3: DeprecationWarning: `langchain-community`
is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-
community/issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.vectorstores import FAISS
```

Step 3: Load the Document

What are we performing?

- Reading the text file.
- Storing its contents in the variable `data`.

Why are we performing it?

- The text file acts as the knowledge source for the RAG system.

In [3]:
```
data = open(r'D:\qsp GenAI\RAG\machine_learning_2000_sentences.txt').read()
# data
```

$$ Text Normalization $$

Step 4: Convert to Lowercase

What are we performing?

- Preparing the document for preprocessing.
- Converting all characters into lowercase.

Why are we performing it?

- Clean and standardized text improves chunking, embeddings and retrieval.
- Ensures consistent text representation.
- Prevents uppercase and lowercase words from being treated differently.

In [4]:
```
data = data.lower()
```

Step 5: Remove Extra Whitespaces

What are we performing?

- Removing multiple consecutive spaces using Regular Expressions (Regex).

Why are we performing it?

- Produces cleaner and more consistent text.

In [5]:
```
import re
data = re.sub(r'\s{2,}',' ', data)
```

Step 6: Remove Statement Numbers

What are we performing?

- Removing numbering patterns such as `machine learning statement 1:`, `2:`, `3:`.

Why are we performing it?

- Statement numbers do not contribute to the meaning of the document.

In [6]:
```
data[:400]
```

Out[6]:
```
'machine learning statement 1: a workflow emphasizing unsupervised\nlearning can be improved by carefully
validating feature scaling,\nmonitoring logistic regression, documenting assumptions, and comparing\nresults
against a meaningful baseline before deployment. machine\nlearning statement 2: a workflow emphasizing
reinforcement learning can\nbe improved by carefully validating linear regression, monit'
```

In [7]:
```
data = re.sub(r'machine learning statement \d+:','',data)
data = re.sub(r'machine learning\nstatement \d+:','',data)
data = re.sub(r'machine\nlearning statement \d+:','',data)
```

In [8]:
```
data[:1000]
```

Out[8]:
```
' a workflow emphasizing unsupervised\nlearning can be improved by carefully validating feature
scaling,\nmonitoring logistic regression, documenting assumptions, and comparing\nresults against a meaningful
baseline before deployment.  a workflow emphasizing reinforcement learning can\nbe improved by carefully
validating linear regression, monitoring\ndimensionality reduction, documenting assumptions, and comparing
results\nagainst a meaningful baseline before deployment.  a workflow emphasizing classification can be improved
by\ncarefully validating neural networks, monitoring randomizedsearchcv,\ndocumenting assumptions, and comparing
results against a meaningful\nbaseline before deployment.  a workflow\nemphasizing regression can be improved by
carefully validating t-sne,\nmonitoring reinforcement learning, documenting assumptions, and\ncomparing results
against a meaningful baseline before deployment.\n a workflow emphasizing clustering can be\nimproved by
carefully validating normalization, moni'
```

Step 7: Expand Contractions

What are we performing?

- Importing the `contractions` library.
- Expanding all contractions in the document.

Why are we performing it?

- Used to expand contracted words into their complete form.
- Produces standardized text before further preprocessing.

In [9]:
```
import contractions
data = contractions.fix(data)
```

Step 8: Remove Punctuation

What are we performing?

- Importing punctuation characters.
- Removing punctuation from the document.

Why are we performing it?

- Keeps only meaningful words for embedding generation.

In [10]:
```
data = re.sub(r'[^0-9a-zA-Z\s]','',data)
```

Step 9: Spell Correction using TextBlob

What are we performing?

- Importing the TextBlob library.

Why are we performing it?

- Can be used to correct spelling mistakes before generating embeddings.

In [11]:
```
# from textblob import TextBlob
# data = str(TextBlob(data).correct())
```

Step 10: Load spaCy for Lemmatization

What are we performing?

- Importing spaCy.
- Loading the English language model.

Why are we performing it?

- Required for tokenization, stop-word removal and lemmatization.

In [12]:
```
nlp = spacy.load('en_core_web_sm')
```

Step 11: Perform Tokenization and Lemmatization

What are we performing?

- Tokenizing the document.
- Removing stop words.
- Converting words to their root form (lemma).

Why are we performing it?

- Reduces unnecessary words.
- Produces a clean document before chunking.

In [13]:
```
tokens = nlp(data)
updated_tokens = [token.lemma_ for token in tokens if not token.is_stop]
```

Step 12: Convert Tokens Back to Text

What are we performing?

- Joining all processed tokens into a single string.

Why are we performing it?

- Creates the final cleaned document for chunking.

In [14]:
```
data = ' '.join(updated_tokens).strip()
```

Step 13: Create the Text Splitter

What are we performing?

- Creating a Recursive Character Text Splitter.

Why are we performing it?

- Splits large documents into smaller chunks for retrieval.

In [15]:
```
splitter = RecursiveCharacterTextSplitter(
    chunk_size = 100,
    chunk_overlap = 20
)
```

Step 14: Split the Document into Chunks

What are we performing?

- Splitting the cleaned document into LangChain `Document` objects.

Why are we performing it?

- Each chunk stores both the text and its metadata.

> **Difference from Previous Notebook**
>
> - Previous notebook:
> ```python
> chunks = list(set(splitter.split_text(data)))
> ```
>
> - Current notebook:
> ```python
> chunks = splitter.create_documents([data])
> ```
>
> - `create_documents()` returns `Document` objects instead of plain strings.

In [16]:
```
chunks = splitter.create_documents([data])
```

Step 15: Display the First Chunk

What are we performing?

- Printing the first chunk.

Why are we performing it?

- To verify that chunking was performed correctly.

In [17]:
```
print(chunks[0])
```

```
page_content='workflow emphasizing unsupervise
 learning improve carefully validate feature scale'
```

Step 16: Check the Chunk Datatype

What are we performing?

- Checking the datatype of the first chunk.

Why are we performing it?

- Confirms that each chunk is stored as a LangChain `Document`.

In [18]:
```
type(chunks[0])
```

Out[18]:
```
langchain_core.documents.base.Document
```

Step 17: Display the Chunk Content

What are we performing?

- Printing only the text stored inside the first chunk.

Why are we performing it?

- Displays the actual document content without metadata.

In [19]:
```
print(chunks[0].page_content)
```

```
        workflow emphasizing unsupervise
         learning improve carefully validate feature scale
```

Step 18: Add Metadata to the Chunk (Method 1)

What are we performing?

- Assigning metadata directly to the chunk.

Why are we performing it?

- Helps identify the source document during retrieval.

> **Difference from Previous Notebook**
>
> - Metadata was not available because chunks were stored as strings.

Step 19: Add Metadata as a Dictionary (Recommended)

What are we performing?

- Storing metadata as a dictionary.

Why are we performing it?

- Allows multiple metadata fields such as filename, author, source, etc.

> **Difference from Previous Step**
>
> - Previous step stored only a string.
> - This step stores structured metadata as key-value pairs.

In [20]:
```
# chunks[0].metadata = 'data.txt'
# chunks
chunks[0].metadata = {'file_name':'data.txt'}
# chunks
```

Step 20: Chunk Embeddings (Convert Chunks to vectors)

What are we performing?

- Initializing the Hugging Face embedding model.
- Preparing to convert text chunks into vector embeddings.

Why are we performing it?

- Converts every document chunk into vector embeddings.
- Vector embeddings are required for semantic similarity search.

In [21]:
```
embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-miniLM-L6-V2"
)
```

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 1991.33it/s]
```

Step 21: Create the LangChain FAISS Vector Database

What are we performing?

- Creating a FAISS vector database from all document chunks.

Why are we performing it?

- Stores vector embeddings for fast semantic similarity search.

> **Difference from Previous Notebook**
>
> - Previous notebook manually:
> - Generated embeddings
> - Normalized vectors
> - Created `IndexFlatIP`
> - Added vectors to FAISS
>
> - Current notebook performs all these steps using:
> `python
> FAISS.from_documents()
> `

In [22]:
```
vector_db = FAISS.from_documents(
    documents = chunks,  # stores all document chunks
    embedding = embedding_model
)
```

Step 22: Perform Similarity Search

What are we performing?

- Searching the vector database using the user's query.

Why are we performing it?

- Retrieves the document chunks that are most similar to the query.

> **Difference from Previous Notebook**
>
> - Previous notebook used:
> `python
> index_faiss_db.search()
> `
>
> - Current notebook uses:
> `python
> vector_db.similarity_search()
> `

$$ Retrieval Stage $$

In [23]:
```
user_query = 'What is Machine Learning'
r_chunks = vector_db.similarity_search(user_query) # returns directly content, not distance or index
```

Step 23: Display Retrieved Chunks

What are we performing?

- Printing the content of every retrieved chunk.

Why are we performing it?

- To verify that relevant information has been retrieved.

In [38]:
```
# r_chunks = set()
for chunk in chunks:
    # print(chunk.page_content)
```

```
  Cell In[38], line 3
    # print(chunk.page_content)
                                ^
_IncompleteInputError: incomplete input
```

Step 24: Combine Retrieved Chunks

What are we performing?

- Extracting the text (`page_content`) from each retrieved document.
- Removing duplicate chunks.
- Combining all retrieved text into a single string.

Why are we performing it?

- Creates the final context that will be passed to the LLM during the Generation stage.

In [25]:
```
updated_r_chunks = set()
for chunk in chunks:
    updated_r_chunks.add(chunk.page_content)

R_text = '\n'.join(updated_r_chunks)
```

Step 25: Display the Retrieved Context

What are we performing?

- Displaying the final retrieved context.

Why are we performing it?

- Verifies the context before sending it to the language model.
- This is the final output of the Retrieval stage.

In [26]:
```
R_text[:500]
```

Out[26]:
```
'improve carefully validate anomaly detection monitor \n umap document assumption compare result\nknearest
neighbor document assumption compare result\ncompare result meaningful baseline deployment \n  workflow
emphasizing transformer\nimprove carefully validate normalization monitor naive baye\nemphasizing pca improve
carefully validate\ncompare result meaningful baseline deployment \n  workflow emphasizing model
evaluation\nmachine monitor cross validation documenting assumption\nimprove carefully valida'
```

LangChain Notes

if we use llms using langchain, we can use pipeline also
- by using langchain we can create ai agents also.
- we use langchain and langraph to create ai agent
- we won't use huggingface, because we will reach limit. in claude also
- we will download llm models.

$$ Generation Stage $$

- Retrieval only returns relevant context.
- Generation converts the retrieved context into a meaningful answer.

Step 26: Simplified Retrieval Function(not necessarily important b/c retrieval part already don)

What are we performing?

- Creating a retrieval function using LangChain FAISS.

Why are we performing it?

- Retrieves relevant document chunks with much less code.

> **Difference from Previous Notebook**
>
> - Replaces manual embedding generation and `index.search()` with `vector_db.similarity_search()`.

Step 27: Create the Generation Function

What are we performing?

- Creating a function to generate the final answer using Hugging Face.

Why are we performing it?

- Combines the retrieved context and the user query.
- Sends both to an LLM to generate the final response.

> **Difference from Previous Notebook**
>
> - Retrieval is now handled by LangChain.
> - Only the generation logic needs to communicate with the LLM.

In [32]:
```
def r_search(query, k=2):

    R_chunks = vector_db.similarity_search(query, k=k)

    R_chunks = {doc.page_content for doc in R_chunks}

    R_Text = '\n'.join(R_chunks)

    return R_Text



def g_text(r_search, query):
    import os
    prompt = f'''
            You're an helpful assistant
            Assigned Task for you.
            Structure my output => {r_search} for this input => {query}
            Output Structure:
            Input: {query}
            Output: structured output
'''
    llm_model = ChatGoogleGenerativeAI(model="gemini-3.5-flash",
                                       api_key=os.environ["GEMINI_API_KEY"],
                                       timeout=60)
    response = llm_model.invoke(prompt).content
    return response

user_prompt = 'What is Machine Learning?'
user_prompt = re.sub(r'[0-9a-zA-Z\s]','',data)
r_response = r_search(user_prompt)
g_response = g_text(r_response, user_prompt)
print(g_response[0]['text'])
```

Input:
How do we make SVM better? Well, SVMs are good but they need tuning. First, you have to scale your features, like using StandardScaler, because SVM is sensitive to distance. Also, choosing the right kernel is key - linear, RBF, polynomial. If you use RBF, you have to tune C and Gamma. C controls the trade-off between smooth decision boundary and classifying training points correctly. Gamma defines how far the influence of a single training example reaches. We also need to handle imbalanced data using class_weight='balanced'. Cross-validation is important to avoid overfitting. Feature selection helps too.

***

Output: structured output

Guide to Optimizing Support Vector Machine (SVM) Performance

To maximize the predictive power and generalization capability of a Support Vector Machine (SVM), a systematic approach to optimization must be applied. Below is the structured roadmap to carefully improve SVM models.

---

1. Essential Data Preprocessing
SVMs maximize the margin between support vectors based on distance metrics. Therefore, preprocessing is the most critical first step.

*   **Feature Scaling (Mandatory):**
    *   *Action:* Apply **Standardization (Z-score scaling)** or **MinMax Scaling** to all numerical features.
    *   *Why:* SVMs are highly sensitive to the scale of input features. Unscaled features with large ranges will dominate the distance calculations, rendering the model ineffective.
*   **Handling Imbalanced Datasets:**
    *   *Action:* Set the hyperparameter `class_weight='balanced'` or use sampling techniques like **SMOTE** (Synthetic Minority Over-sampling Technique).
    *   *Why:* Standard SVMs maximize overall accuracy, which can lead to a bias toward the majority class in imbalanced datasets.

---

2. Kernel Selection & Hyperparameter Tuning
The choice of kernel and hyperparameters dictates the decision boundary shape and model complexity.

A. Kernel Selection
Choose the mathematical function used to map data into higher-dimensional spaces:
*   **Linear Kernel:** Best for linearly separable data or high-dimensional text classification.
*   **Radial Basis Function (RBF) Kernel:** The default choice for non-linear data.
*   **Polynomial Kernel:** Useful for image processing or when degree interactions are known.

B. Careful Hyperparameter Tuning (Grid Search / Randomized Search)
When using the popular **RBF Kernel**, you must carefully balance two highly interactive parameters:

Hyperparameter	Low Value Effect	High Value Effect	Key Recommendation
C (Regularization)	**Underfitting:** Allows more misclassifications for a smoother, simpler decision boundary (high bias, low variance).	**Overfitting:** Forces the model to classify all training points correctly, leading to a complex boundary (low bias, high variance).	Tune C exponentially (e.g., 10^{-3} to 10^{3}) using cross-validation to find the sweet spot.

| **γ (Gamma - Kernel Coefficient)** | **Underfitting:** Large radius of influence; the decision boundary is too smooth and fails to capture local patterns. | **Overfitting:** Tiny radius of influence; the decision boundary becomes highly irregular and creates "islands" around individual points. | Tune γ carefully alongside C. A high γ almost always requires a lower C to prevent overfitting. |

---

3. Dimensionality Reduction & Feature Selection
While SVMs handle high-dimensional spaces relatively well, irrelevant features introduce noise.

*   **Feature Selection:** Use techniques like **Recursive Feature Elimination (RFE)** or tree-based feature importances to remove redundant variables.
*   **Dimensionality Reduction:** Apply **Principal Component Analysis (PCA)** prior to training the SVM to reduce computational complexity and mitigate the "curse of dimensionality."

---

4. Robust Validation Strategy
To guarantee that improvements are genuine and not just memorization of the training set:

*   **K-Fold Cross-Validation:** Always evaluate hyperparameter combinations using stratified k-fold cross-validation.
*   **Pipeline Integration:** Ensure that feature scaling is fit *only* on the training folds within the cross-validation loop to prevent data leakage.

Only generation function

In [34]:
```
# Import the os module to access environment variables
import os
# Function to perform Retrieval + Generation
def generate_response(user_query):
    # ---------------------- Retrieval ----------------------

    # Retrieve the most relevant document chunks from the FAISS vector database
    r_chunks = vector_db.similarity_search(user_query)

    # Extract only the text (page_content) from each retrieved Document object
    # Remove duplicate chunks using set()
    # Join all chunks into a single context string
    context = "\n".join(
        list(set(chunk.page_content for chunk in r_chunks))
    )

    # ---------------------- Prompt ----------------------
    # Create the prompt that will be sent to the language model
    prompt = f"""
            You're an helpful assistant
            Assigned Task for you:
            Structure my output => Context: {context}
            for this input => Question: {user_query}

            Answer:
    """
    llm_model = ChatGoogleGenerativeAI(
        model="gemini-3.5-flash", # 3.5 flash 2.0 flash, 2.5-pro, 3.6-flash
    )

    response = llm_model.invoke(prompt).content
    return response

# User's question
user_query = "Explain Machine Learning"

# Generate the response using the RAG Pipeline
answer = generate_response(user_query)

# Display the final answer
print(answer[0]['text'])
```

```
Here is a structured explanation of Machine Learning, designed to highlight the powerful paradigm of **Support
Vector Machines (SVM)** and the advanced tuning capabilities of **Bayesian Optimization** to carefully improve
model performance.

---

# Understanding Machine Learning

At its core, **Machine Learning (ML)** is a subset of artificial intelligence that enables computers to learn
from data and make predictions or decisions without being explicitly programmed. Instead of writing rigid, rule-
based code, we feed data into ML algorithms, which dynamically build mathematical models to identify patterns.

While ML spans many techniques, achieving peak performance requires choosing the right algorithms and **carefully
improving** them through systematic optimization.

---

## 1. The Core Paradigm: Support Vector Machines (SVM)

To understand how ML algorithms classify complex data, we look to the **Support Vector Machine (SVM)**—one of the
most robust and mathematically elegant supervised learning algorithms.

### How SVM Works
An SVM finds the optimal boundary (called a **hyperplane**) that separates different classes of data.
* **The Margin:** Unlike algorithms that just find any boundary, SVM aims for the *maximum margin*—the largest
possible distance between the hyperplane and the nearest data points of any class.
* **Support Vectors:** These nearest data points are called "support vectors." They are critical because they
define the position and orientation of the boundary.

### Improving SVM Carefully
SVMs are incredibly powerful, but to prevent overfitting and ensure generalization, they must be **improved
carefully** through precise parameter tuning:
* **The Kernel Trick:** Standard SVMs find linear boundaries. For non-linear data, SVMs use "kernels" (like the
Radial Basis Function - RBF) to project data into higher dimensions where it becomes linearly separable. Choosing
the right kernel requires careful analysis.
* **The $C$ Parameter (Regularization):** This controls the trade-off between a smooth decision boundary and
classifying training points correctly. A value of $C$ that is too high overfits the data, while a value too low
underfits it.
```

* **The Gamma (γ) Parameter:** For non-linear kernels, gamma defines how far the influence of a single training example reaches. Tuning this carefully is crucial for model stability.

---

2. The Tuning Masterclass: Bayesian Optimization

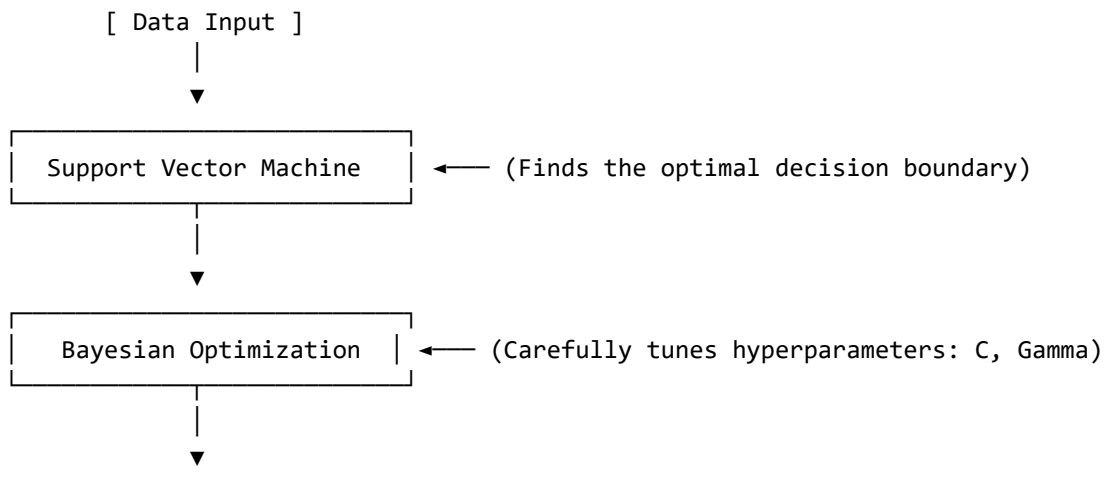
Once we have a powerful model like an SVM, how do we find the absolute best hyperparameters (like C, gamma, and the choice of kernel)? Standard trial-and-error (Grid Search) is slow and inefficient.

To **improve our models carefully and efficiently**, we use **Bayesian Optimization**.

What is Bayesian Optimization?
Bayesian Optimization is an advanced, sequential design strategy for global optimization. It is specifically designed to find the best hyperparameters for black-box functions that are computationally expensive to evaluate.

How It Carefully Improves Models
Rather than guessing randomly, Bayesian Optimization treats hyperparameter tuning as a smart, iterative process:
1. **The Surrogate Model (Gaussian Process):** It builds a probabilistic model of the objective function (e.g., the SVM's validation accuracy based on different hyperparameters). This model estimates the performance of the hyperparameters we haven't tested yet, along with the uncertainty of those estimates.
2. **The Acquisition Function:** This mathematical tool decides where to sample next. It carefully balances two forces:
   * **Exploitation:** Searching in areas known to yield high performance.
   * **Exploration:** Searching in areas with high uncertainty to find potentially better hidden setups.
3. **Meticulous Iteration:** With each trial, the system updates its internal beliefs, narrowing down the optimal hyperparameter values with minimal computational cost.

Summary: The Synergy of ML



By combining robust algorithms like **Support Vector Machines** with rigorous, intelligent tuning frameworks like **Bayesian Optimization**, machine learning transitions from a process of guesswork to a highly disciplined, carefully improved science of predictive modeling.

In [ ]:
```
# pip install google-generativeai
```

In [ ]:
```
# import google.generativeai as genai
# import os

# genai.configure(api_key=os.environ["GEMINI_API_KEY"])

# for model in genai.list_models():
#     if "generateContent" in model.supported_generation_methods:
#         print(model.name)
```

```
C:\Users\LENOVO\AppData\Local\Temp\ipykernel_17556\3099384382.py:1: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  import google.generativeai as genai
```

```
models/gemini-2.5-flash
models/gemini-2.5-pro
models/gemini-2.0-flash
models/gemini-2.0-flash-001
models/gemini-2.0-flash-lite-001
models/gemini-2.0-flash-lite
models/gemini-2.5-flash-preview-tts
models/gemini-2.5-pro-preview-tts
models/gemma-4-26b-a4b-it
models/gemma-4-31b-it
models/gemini-flash-latest
models/gemini-flash-lite-latest
models/gemini-pro-latest
models/gemini-2.5-flash-lite
models/gemini-2.5-flash-image
models/gemini-3-pro-preview
models/gemini-3-flash-preview
models/gemini-3.1-pro-preview
models/gemini-3.1-pro-preview-customtools
models/gemini-3.1-flash-lite-preview
models/gemini-3.1-flash-lite
models/gemini-3-pro-image-preview
models/gemini-3-pro-image
models/nano-banana-pro-preview
models/gemini-3.1-flash-image-preview
models/gemini-3.1-flash-image
models/gemini-3.1-flash-lite-image
models/gemini-3.5-flash
models/gemini-3.5-flash-lite
models/gemini-omni-flash-preview
models/gemini-3.6-flash
models/lyria-3-clip-preview
models/lyria-3-pro-preview
models/gemini-3.1-flash-tts-preview
models/gemini-robotics-er-1.5-preview
models/gemini-robotics-er-1.6-preview
models/gemini-robotics-er-2-preview
models/gemini-2.5-computer-use-preview-10-2025
models/antigravity-preview-05-2026
models/deep-research-max-preview-04-2026
models/deep-research-preview-04-2026
models/deep-research-pro-preview-12-2025
```